

目录



- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

Search

原创

Agent 直连你日常使用的 Chrome 浏览器，天然携带登录态。

大家好，我是二哥呀。

上一期我们给 PaiCLI 接上了 Chrome DevTools MCP，Agent 终于能开浏览器了。能导航页面、截图、拿 DOM 快照，微信公众号文章也能读了。

但存在另外一个问题。



每次 Agent 打开的都是一个“新”的 Chrome，没有登录态，没有 Cookie。你让它去看 GitHub 私

有仓库的 README，它无能为力，因为需要登录。

并且每次都要重新输入用户名和密码，这就很受不了。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

这一期，我们来解决这个问题：让 Agent 直连你日常使用的 Chrome，复用你已有的登录状态，就像你本人一样浏览任何需要登录的页面。

01、为什么要复用登录态

先回顾一下上一期的架构。

chrome-devtools-mcp 默认用 `--isolated=true` 启动，每次都会创建一个临时的 `user-data-dir`，浏览器关了临时目录就清掉了。

这种隔离机制安全是安全，但也意味着 Agent 的浏览器和你日常用的 Chrome 完全不是一回事。你的 GitHub 登录态、飞书的 session、公司内网的 SSO 令牌，它一个都不能用。

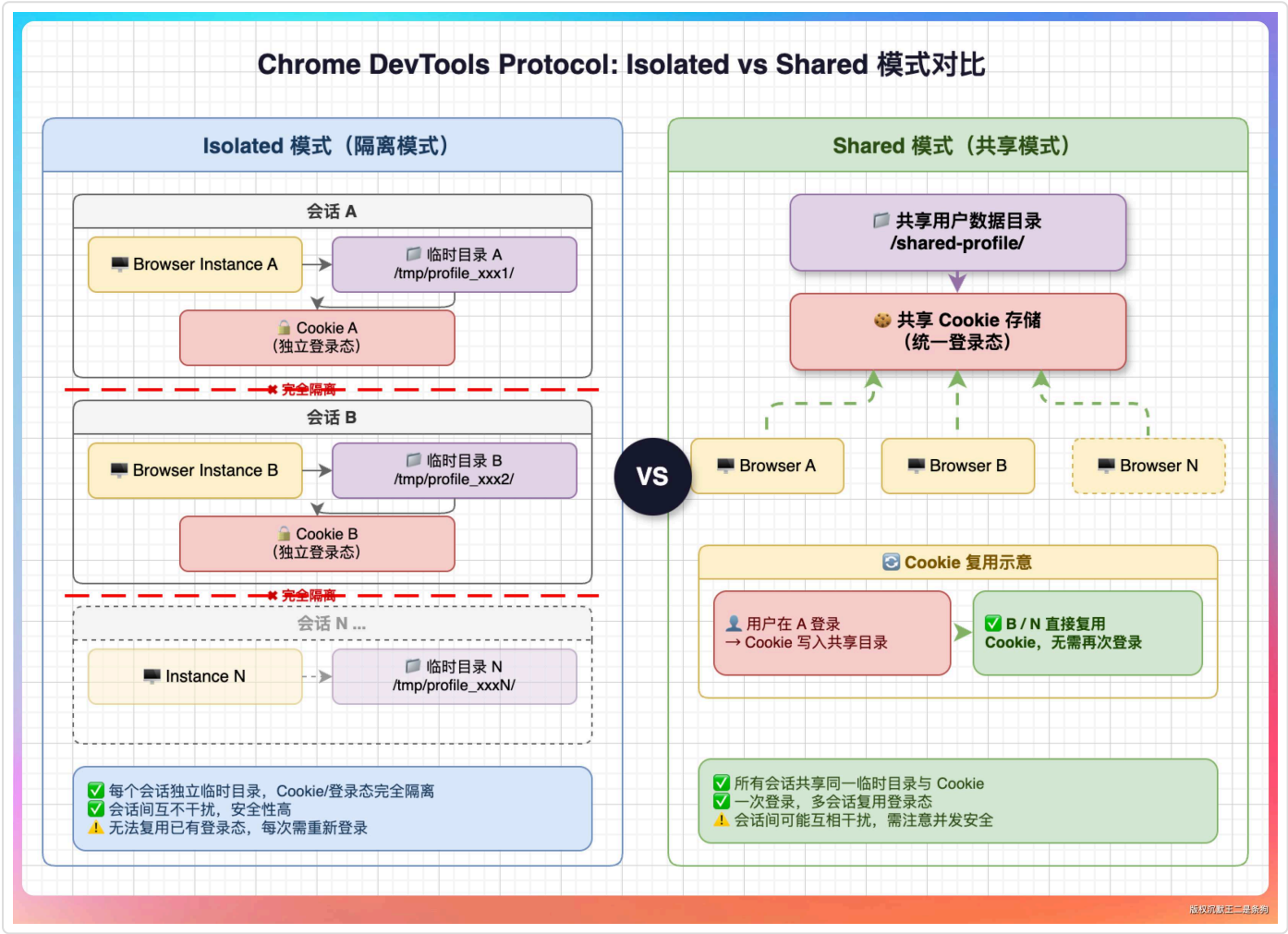
实际开发中会碰到大量需要登录态的场景：查看 GitHub 私有仓库的代码、浏览公司内部的 Wiki、读飞书云文档里的需求说明、查看 Grafana 的监控面板等等。

这些场景有一个共同特点：访问需要身份认证。

Agent 没有登录态，等于一大半的能力被封印了。

目录

- 01、为什么要复用登录态
- 02、开启 Chrome 远程调试
- 03、Agent 自动切换登录态
- 04、自动切换的设计
- 05、autoConnect 的发现机制
- 06、敏感页面保护机制
- 07、标签页防误关
- 08、提示词升级
- 09、PaiCLI 如何写到简历上



chrome-devtools-mcp 原生支持一个参数 `--browser-url`，可以连接到一个已经在运行的 Chrome 实例。

思路是：你启动Chrome，并且开启调试端口，通过 `--browser-url` 连上去，Agent 就能看到所有的登录态了。

02、开启 Chrome 远程调试

要让 PaiCLI 连上你的 Chrome，第一步是让 Chrome 开放 CDP 调试端口。



这种方式最大的好处是：你当前 Chrome 里已经登录好的 GitHub、飞书、公司内网，所有的 Cookie 和 session 都在，不需要重新登录。PaiCLI 连上来就能直接用你现有的登录态。

03、Agent 自动切换登录态

Chrome 远程调试打开后，回到 PaiCLI。

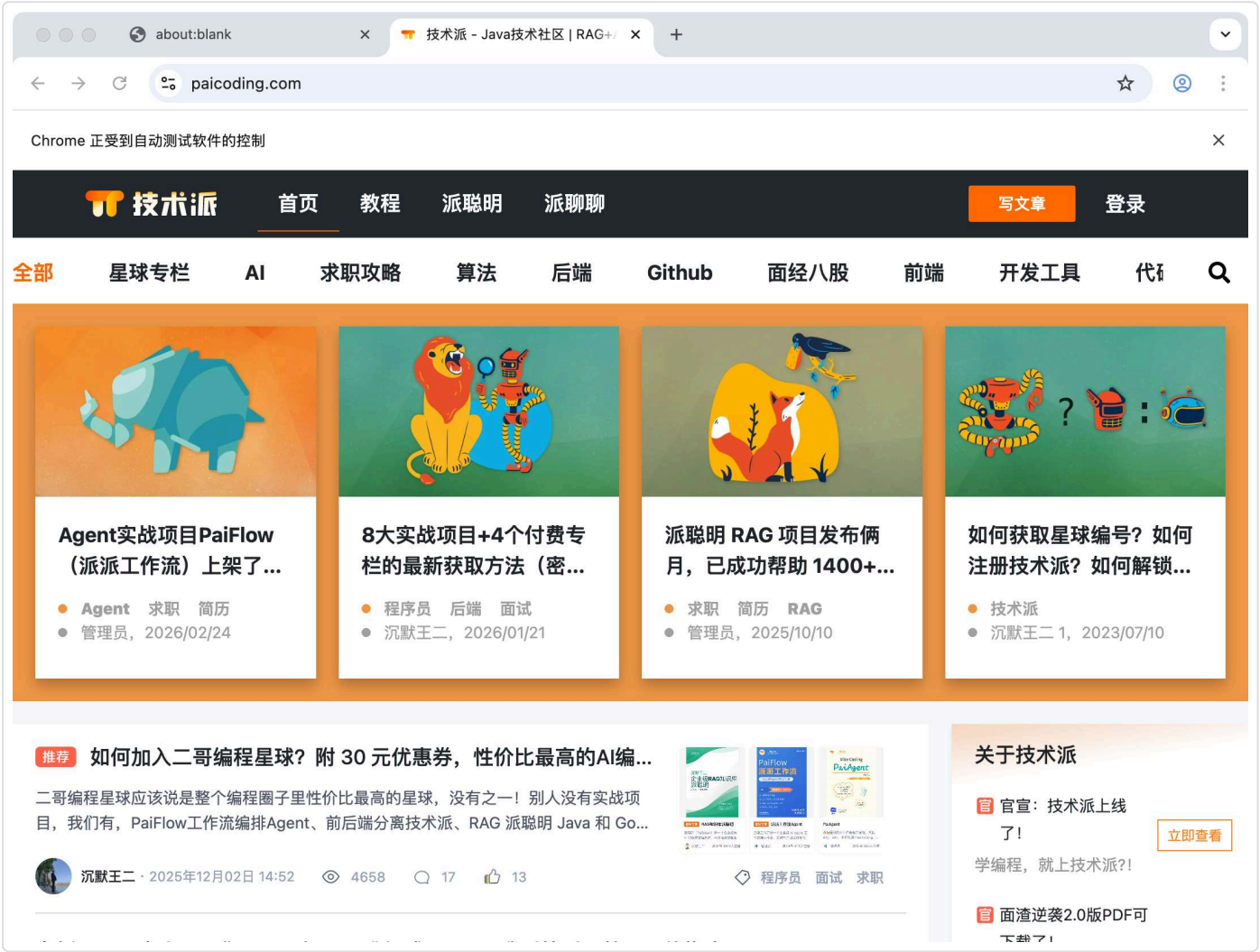
不需要你手动执行连接命令。Agent 会自己判断当前页面需不需要登录态，需要的时候自动切换到 shared 模式，不需要的时候用 isolated 模式。

直接试一下。

让 Agent 去读一个不需要登录的页面：

bash 复制代码

帮我看一下 https://paicoding.com 的首页



Agent 会用 isolated 模式打开一个临时浏览器，拿到页面内容。公开页面不需要登录态，isolated 够用了。

再试一个需要登录的：

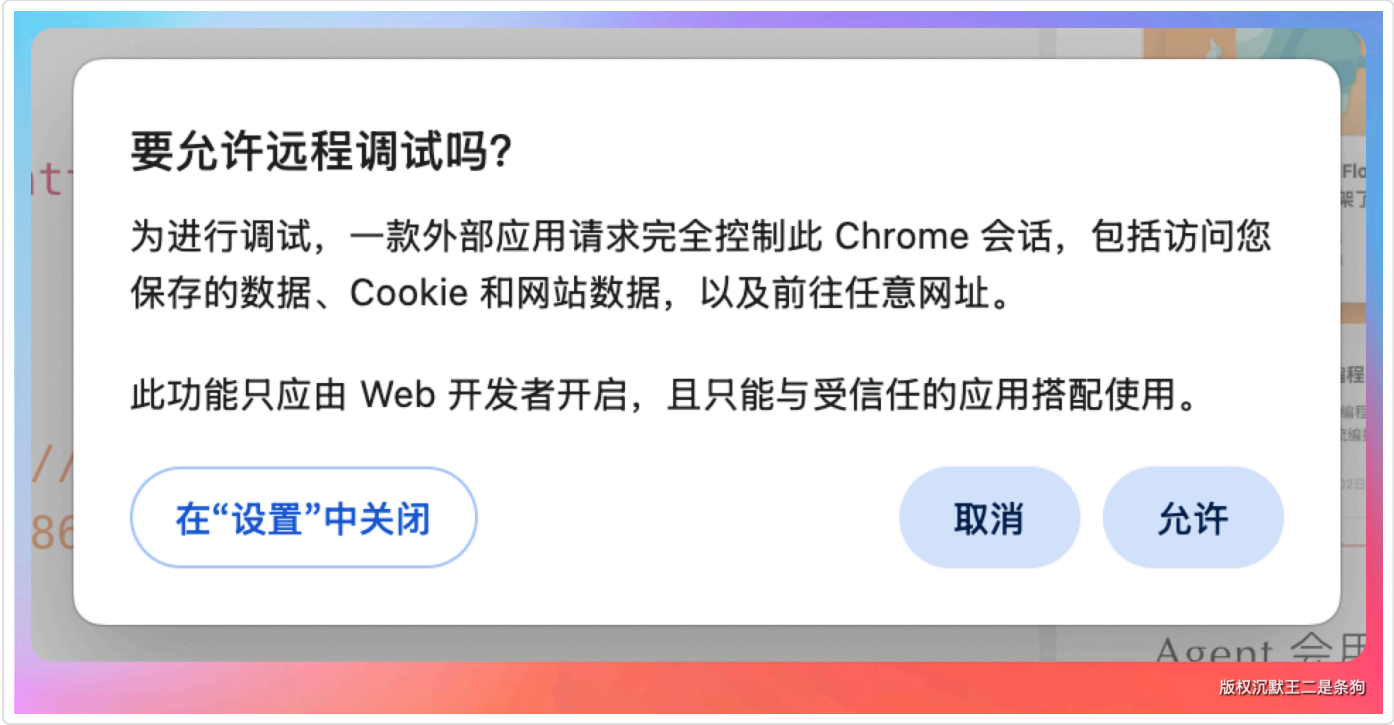
bash 复制代码

帮我看一下语雀上的这篇文档 https://www.yuque.com/itwanger/gykdzg

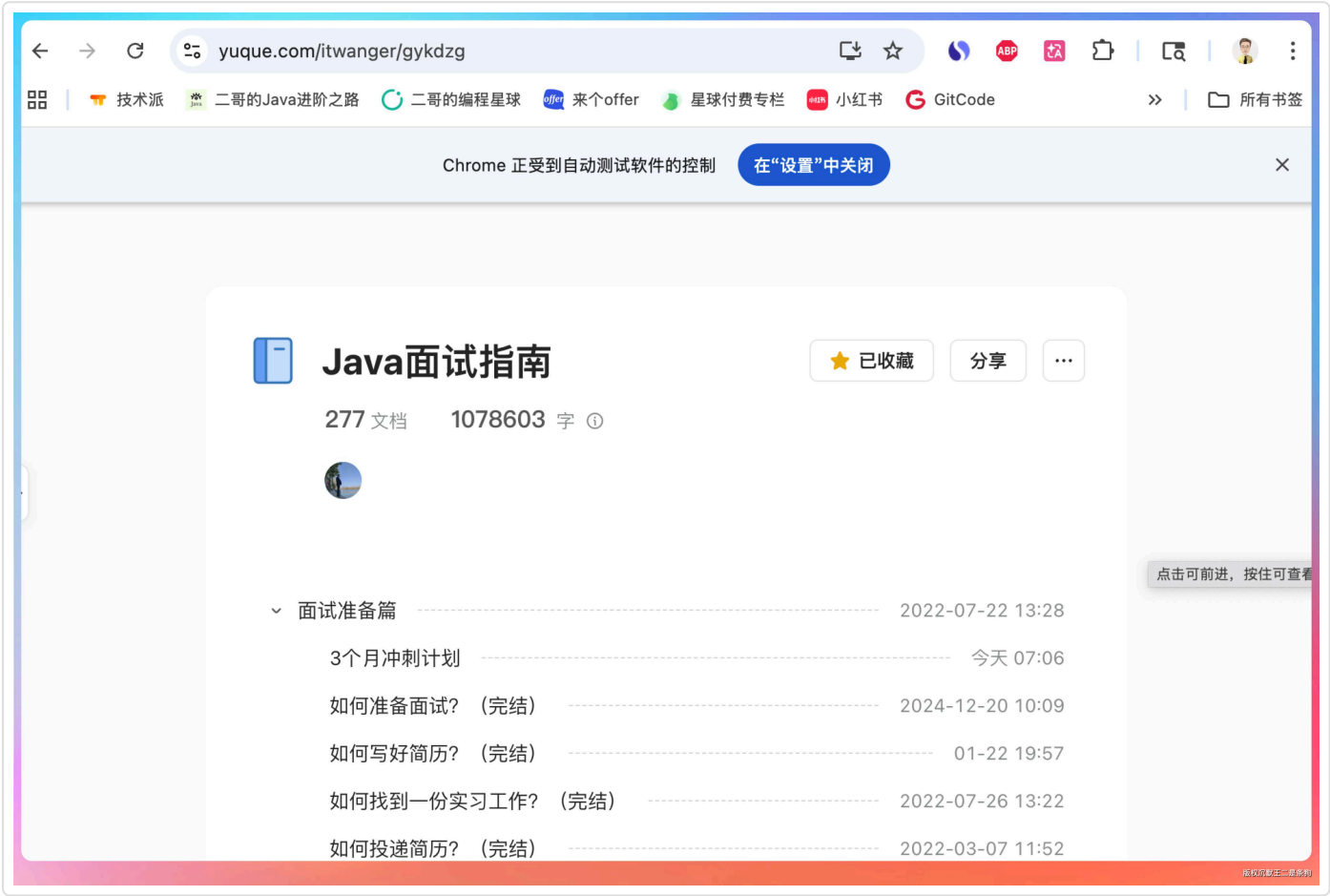
首先会弹出这样一个确认对话框。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)



允许后就会使用你的登录态。

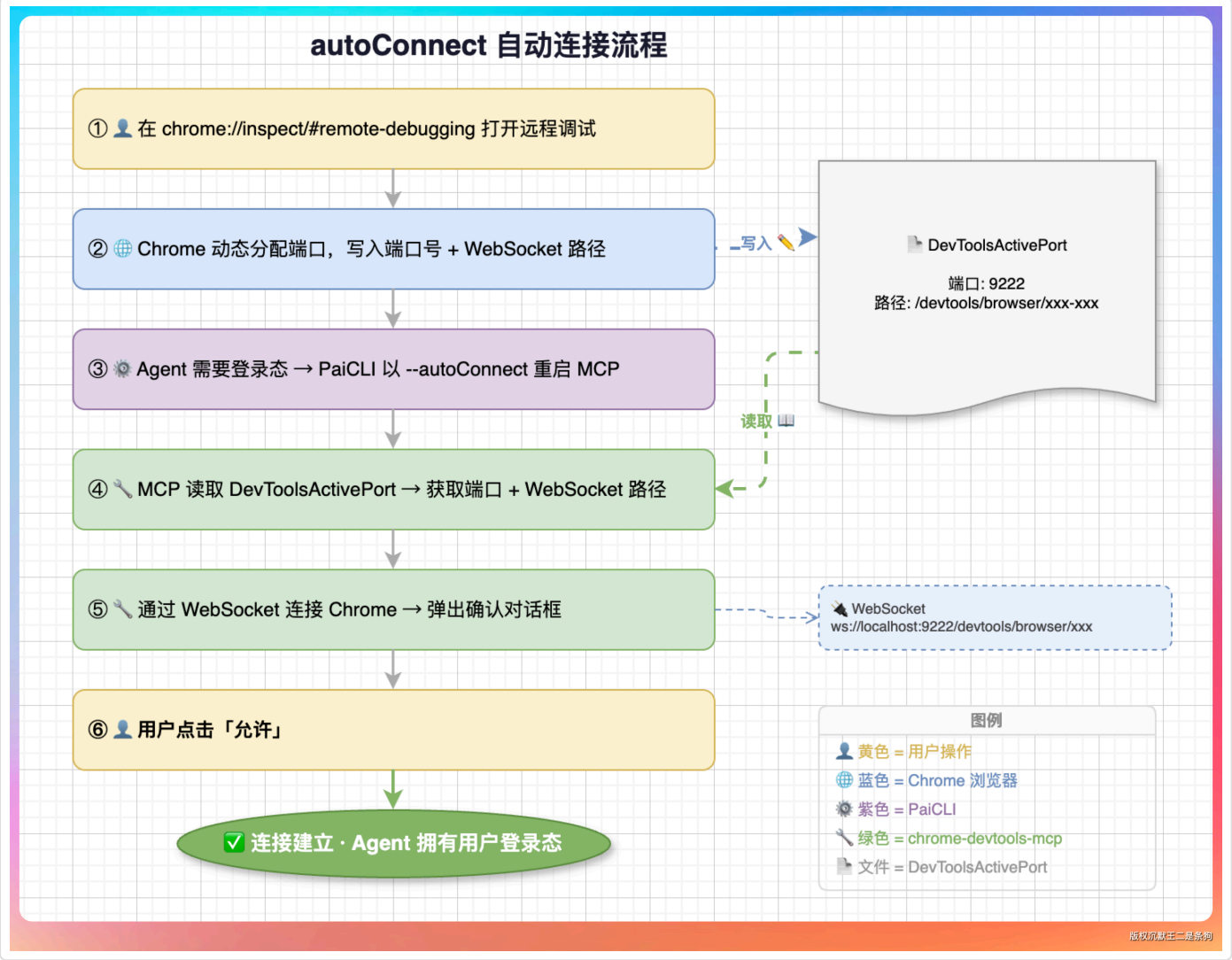


注意右上角，两种模式下的登录用户是不一样的，一个是我已经登录的Chrome账号，另外一个临时用户。

isolated 模式会自动检测本机有没有开启远程调试的 Chrome，发现有，就切换到 shared 模式（`--autoConnect`）重新访问。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)



目录

- 01、为什么要复用登录态
- 02、开启 Chrome 远程调试
- 03、Agent 自动切换登录态
- 04、自动切换的设计
- 05、autoConnect 的发现机制
- 06、敏感页面保护机制
- 07、标签页防误关
- 08、提示词升级
- 09、PaiCLI 如何写到简历上

整个过程你在 Chrome 里能实时看到页面在跳转。

Agent 的思考过程会打印出来：先尝试 isolated，发现需要登录，自动切到 shared 重试。

```
~/Documents/GitHub/paicli git:(main) 13 • +270 -59
java -jar target/paicli-1.0-SNAPSHOT.jar

运行中按 ESC 取消当前任务。
🧠 思考过程
The user wants me to connect to their already-logged-in Chrome browser to access the Yuque document
. I should use the browserconnect tool to reuse their Chrome session.

🤖 回复
🔧 browser_connect x 1
  L {}
好的，我先连接你已登录的 Chrome 浏览器，然后重新访问这个语雀文档。

🔧 调用 MCP 工具 chrome-devtools.new_page
  L {"url":"https://www.yuque.com/itwanger/gykdzg"}

🔧 调用 MCP 工具 chrome-devtools.take_snapshot
  L {}

🤖 回复
🔧 调用 MCP 工具 chrome-devtools.evaluate_script
  L {"function":`() => {\n  // 滚动左侧目录到底，看看有多少文档\n  const panel = document.querySe
...
已成功通过你的 Chrome 登录态访问到这篇语雀文档了！让我再滚动看看有没有更多目录内容。

🔧 调用 MCP 工具 chrome-devtools.take_snapshot
  L {}

🤖 回复
成功通过你的 Chrome 登录态访问到了！以下是这篇语雀文档的详细情况：
```

小红书、语雀、飞书云文档、公司内网系统这些需要登录态的站点，Agent 都能自动处理。你不用关心它在用哪个模式，该切的时候它自己会切。

用 `/browser status` 可以随时查看当前的模式状态：

```
bash 复制代码

/browser status

你: /browser status
浏览器会话
当前模式: shared (复用 autoConnect)
chrome-devtools server: ● ready (29 tools)
旧式 /json/version 探活: ⚠ HTTP 404
自动连接: Chrome 144+ 可在 chrome://inspect/#remote-debugging 勾选 Allow remote debugging 后使用
/browser connect
```

如果你的 Chrome 没有开远程调试，`--autoConnect` 连接失败后 Agent 会继续用 `isolated` 模式，并提示你可以开启远程调试来获得登录态支持。

04、自动切换的设计

这个自动切换的设计经历了几轮迭代，值得展开说说。

最早的方案是让用户手动执行 `/browser connect` 和 `/browser disconnect` 切换。但实际用下来体验很差，用户每次都得想“这个页面需不需要登录态”，想完了还得手动敲命令，用完了还得记着断开。这不是 Agent 该有的体验，Agent 应该自己搞定这些事情。

第二个方案是注册两个 MCP server：一个 `chrome-devtools-isolated`，一个 `chrome-devtools-shared`。Agent 自己选用哪个。但这个方案更离谱，28 个工具变成 56 个，system prompt 的 token 直接翻倍。LLM 看到两套几乎一样的工具。

最终的方案是：**server 名字不变，Agent 在需要登录态时自动切换启动参数，内存里改完重启 server。**

核心实现在 `McpServerManager.restartWithArgs()`：

java 复制代码

```
public synchronized String restartWithArgs(String name, List<String> newArgs) {
    McpServer server = servers.get(name);
    if (server == null) {
        return "✖ 未找到 MCP server: " + name;
    }
    server.config().setArgs(newArgs);
    return restart(name); // 走现有的重启路径
}
```

就这么几行。

改 args，重启，完了。

LLM 看到的工具集始终是这 28 个 `mcp__chrome-devtools__*`，提示词不用改，HITL 逻辑不用改。

Agent 的决策流程是这样的：

第一步，Agent 用 `isolated` 模式打开页面（默认行为，安全）。

第二步，Agent 拿到页面内容后判断是不是登录页。如果是正常内容就直接用，流程结束。

第三步，如果发现是登录页或者权限不足的提示，Agent 调用 `restartWithArgs` 把 `chrome-devtools-mcp` 的启动参数从 `--isolated=true` 切换为 `--autoConnect`。`--autoConnect` 是 `chrome-devtools-mcp` 提供的自动发现机制，它会自动找到本机开启了远程调试的 Chrome 实例并连接上去，不需要指定端口号。连接成功后重新访问同一个页面，连接失败就告诉用户需要开启 Chrome 远程调试。

运作方式

我们在 Chrome M144（目前处于 Beta 版阶段）中添加了一项新功能，可让 Chrome 开发者工具 MCP 服务器请求远程调试连接。这一新流程基于 [Chrome 现有的远程调试功能](#) 构建而成。默认情况下，Chrome 中会停用远程调试连接。开发者必须先前往 `chrome://inspect#remote-debugging` 明确启用此功能。

如果 Chrome DevTools MCP 服务器配置了 `--autoConnect` 选项，则 MCP 服务器将连接到活跃的 Chrome 实例并请求远程调试会话。为避免恶意行为者滥用，每次 Chrome 开发者工具 MCP 服务器请求远程调试会话时，Chrome 都会向用户显示一个对话框，并请求用户授予允许远程调试会话的权限。此外，在调试会话处于活动状态时，Chrome 会在顶部显示“Chrome 正受到自动测试软件的控制”横幅。

远程调试工具栏

切换到 `shared` 模式的时候有一件事必须做：清空 `approvedAllByServer("chrome-devtools")`。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

java 复制代码

```
browserSession.switchToShared("autoConnect");
hitlHandler.clearApprovedAllForServer("chrome-devtools");
```

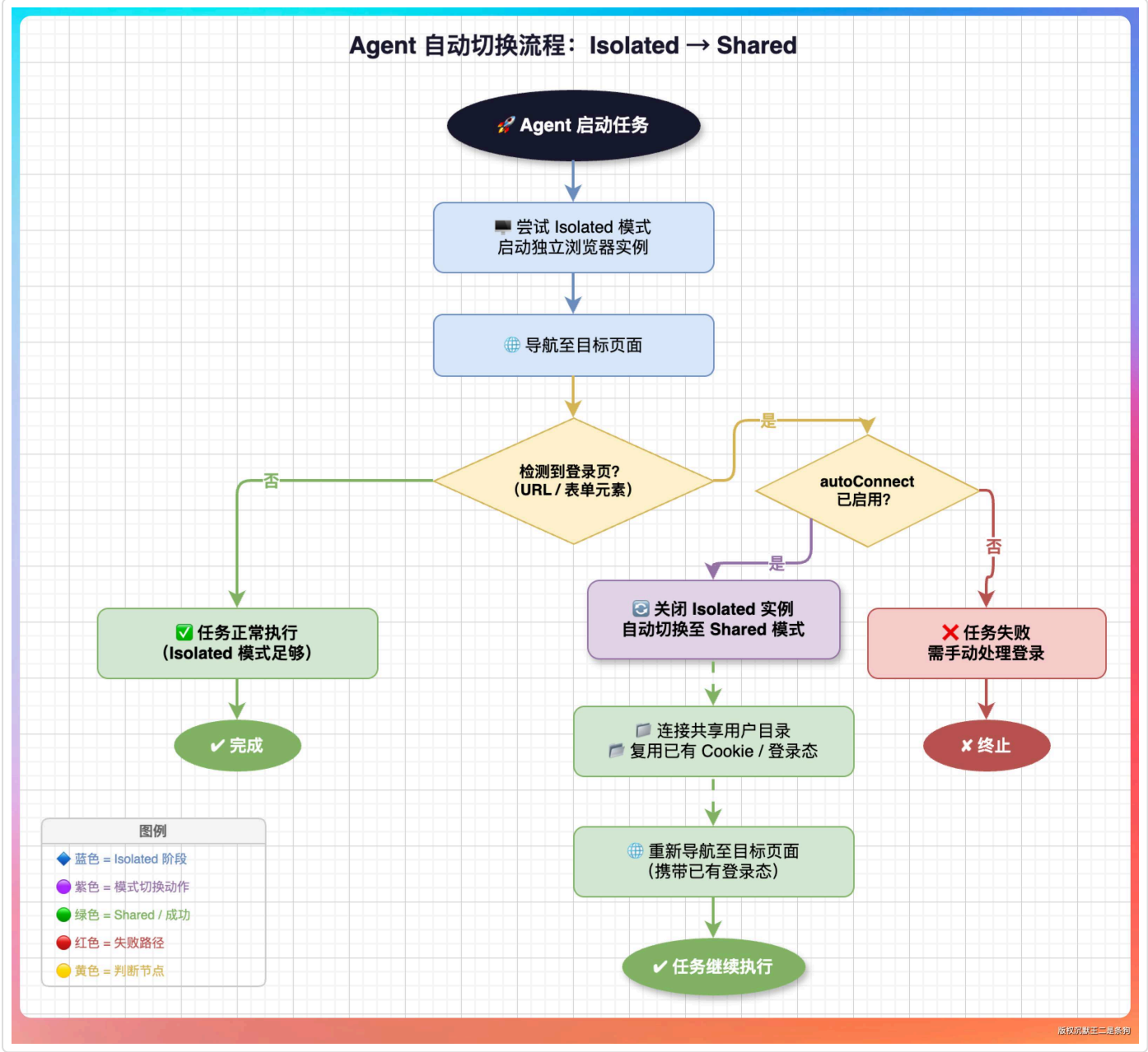
切换只在内存中进行，不会去改你的 `~/.paicli/mcp.json` 文件。

PaiCLI 重启后回到 isolated 模式，默认安全。

你也可以用 `/browser status` 随时查看当前模式，用 `/browser disconnect` 手动切回 isolated。但正常使用根本不需要操心这些，Agent 自己会处理。

目录

- 01、为什么要复用登录态
- 02、开启 Chrome 远程调试
- 03、Agent 自动切换登录态
- 04、自动切换的设计
- 05、autoConnect 的发现机制
- 06、敏感页面保护机制
- 07、标签页防误关
- 08、提示词升级
- 09、PaiCLI 如何写到简历上



05、autoConnect 的发现机制

前面反复提到 `--autoConnect`，Agent 是怎么找到你本机 Chrome 的？

很多人的第一反应是端口扫描——遍历常见端口试一圈，哪个通了就连哪个。chrome-devtools-mcp 没有这么干。端口扫描慢、不可靠，还容易误连到别的服务上去。

它用的是文件发现。

Chrome 在开启远程调试后，会在自己的用户数据目录下写入一个文件：`DevToolsActivePort`。这个文件只有两行：

bash 复制代码

```
52837
/devtools/browser/a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

第一行是 Chrome 动态分配的调试端口号，第二行是 WebSocket 的路径。

chrome-devtools-mcp 拿到这两行，拼成

`ws://127.0.0.1:52837/devtools/browser/a1b2c3d4...`，直接建立 WebSocket 连接。

整个发现过程就是读一个文件，没有网络探测，没有端口扫描。

javascript 复制代码

```
// chrome-devtools-mcp 的连接逻辑（简化）
const portPath = path.join(userDataDir, 'DevToolsActivePort');
const fileContent = await fs.promises.readFile(portPath, 'utf8');
const [rawPort, rawPath] = fileContent
  .split('\n')
  .map(line => line.trim())
  .filter(line => !!line);

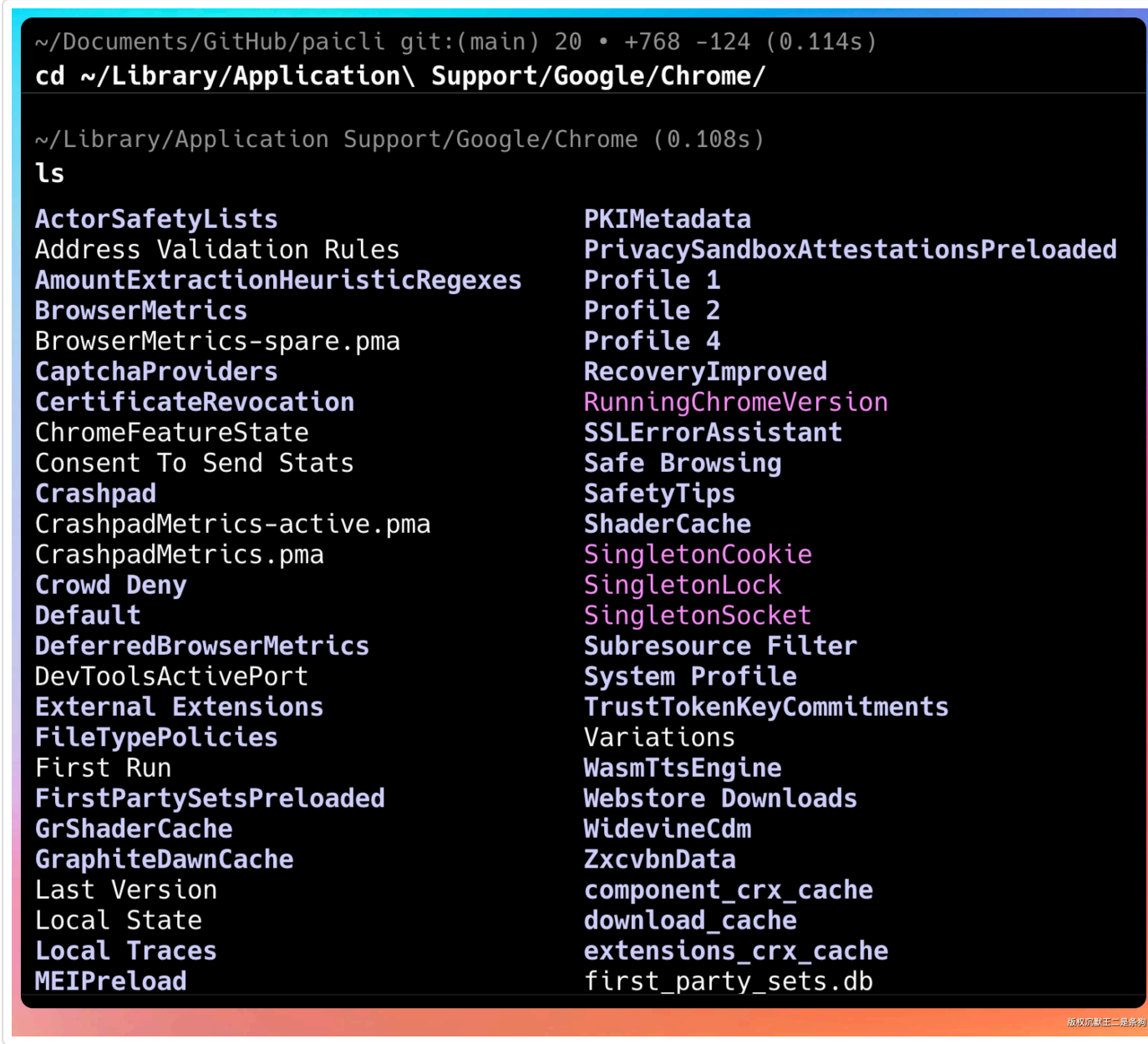
const port = parseInt(rawPort, 10);
const browserWSEndpoint = `ws://127.0.0.1:${port}${rawPath}`;
```

为什么要用文件而不是固定端口？

因为 Chrome 每次开启远程调试时，端口是随机分配的，不是固定的 9222。固定端口容易冲突——你本机跑着好几个 Chrome 实例或者其他开发工具占了 9222，直接就连不上了。随机端口 + 文件记录，Chrome 自己写、MCP 自己读，不会冲突。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

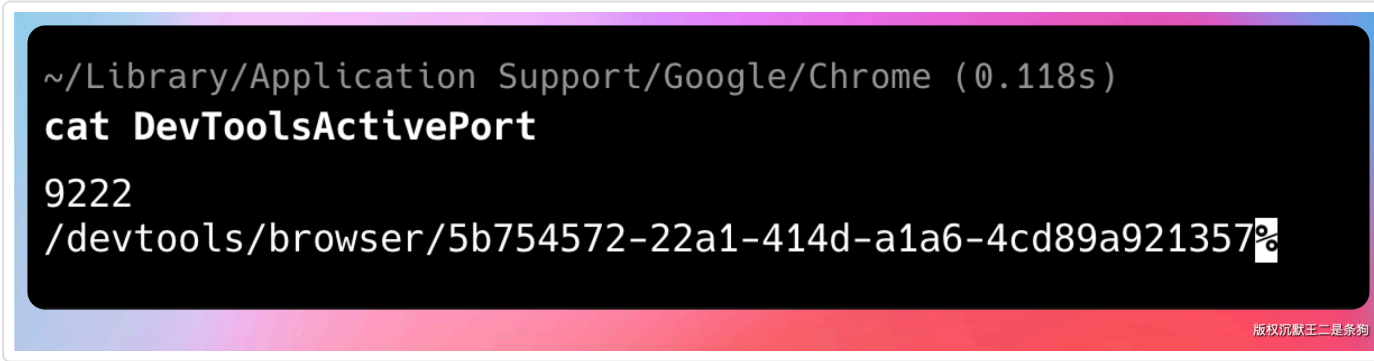


DevToolsActivePort 文件的位置跟操作系统有关：

- **macOS**: ~/Library/Application Support/Google/Chrome/DevToolsActivePort
- **Linux**: ~/.config/google-chrome/DevToolsActivePort
- **Windows**: %LOCALAPPDATA%\Google\Chrome\User Data\DevToolsActivePort

chrome-devtools-mcp 内部是通过 Puppeteer 定位 Chrome 的用户数据目录的，不需要你手动指定路径。

--autoConnect 加上 --channel=stable（默认值），Puppeteer 就知道去哪找。

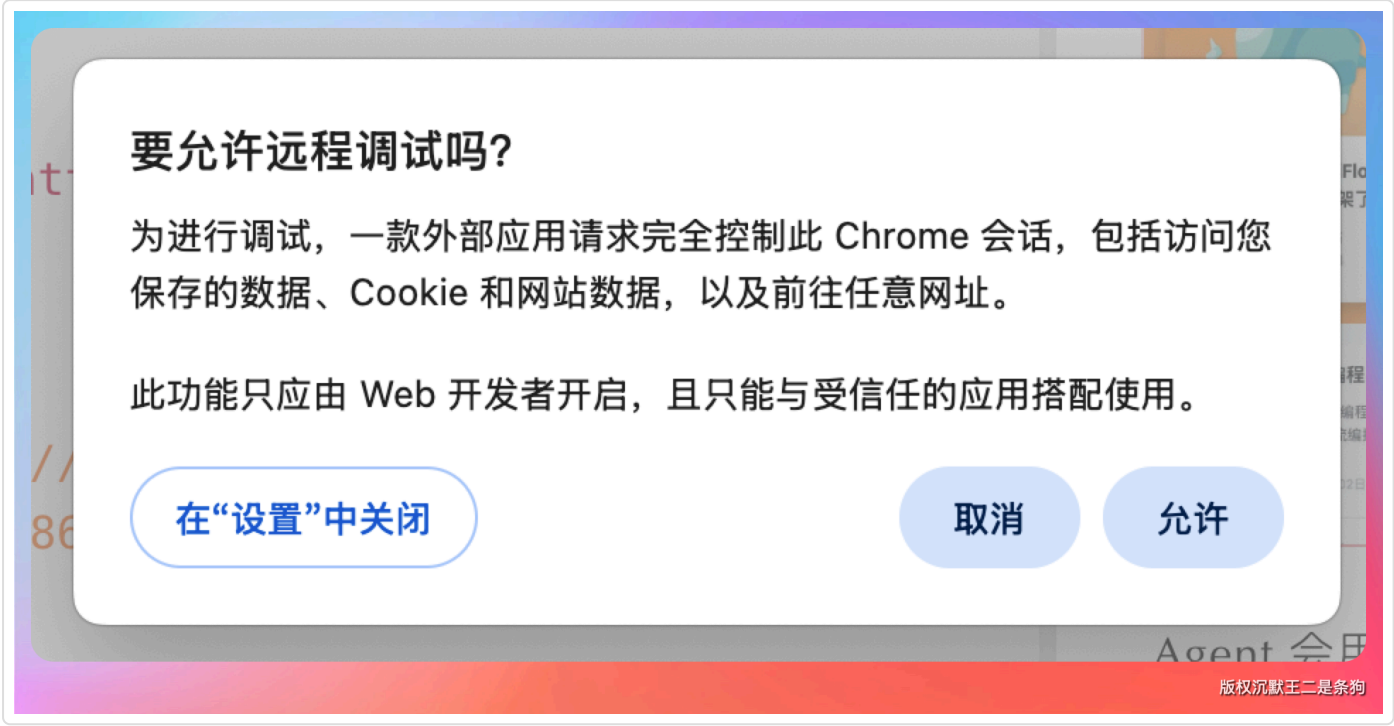


再说连接过程中的权限控制。

你在 `chrome://inspect/#remote-debugging` 打开远程调试的开关后，Chrome 并不是无条件接受所有连接。

当 `chrome-devtools-mcp` 第一次尝试连接时，Chrome 会弹出一个原生的确认对话框，问你是否允许这个外部程序接入。

你点了“允许”，连接才能建立。这是 Chrome 自己的安全机制，不是 PaiCLI 加的。

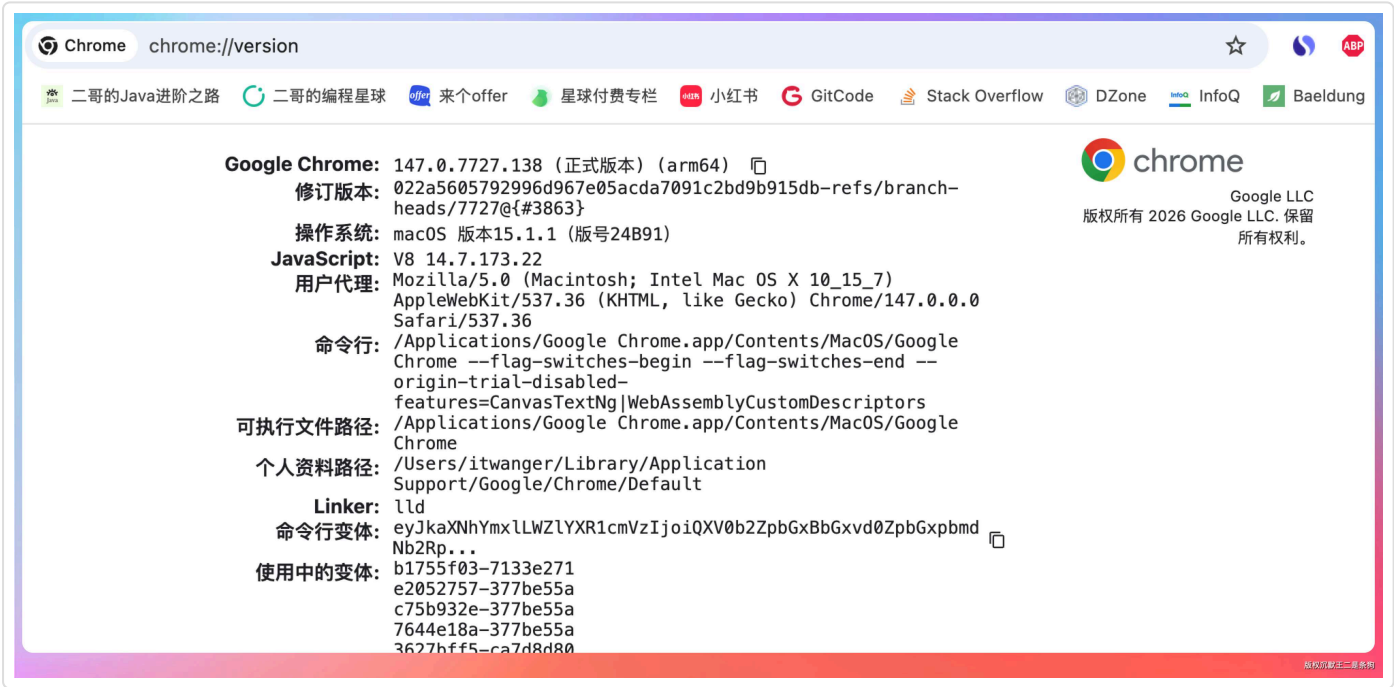


整个流程串起来是这样的：

- 你在 `chrome://inspect/#remote-debugging` 打开远程调试
- Chrome 动态分配端口，把端口号和 WebSocket 路径写入 `DevToolsActivePort` 文件
- Agent 需要登录态时，PaiCLI 用 `--autoConnect` 重启 `chrome-devtools-mcp`
- `chrome-devtools-mcp` 读取 `DevToolsActivePort` 文件，拿到端口和路径
- 通过 WebSocket 连接 Chrome，Chrome 弹出确认对话框
- 用户点“允许”，连接建立，Agent 拥有你的登录态

一个硬性前提：**Chrome 版本必须 144 以上**。`chrome://inspect/#remote-debugging` 这个页面和对应的远程调试权限 UI 是 Chrome 144 引入的。低于这个版本的 Chrome 打开这个地址只会看到空白页。

查看你的 Chrome 版本：地址栏输入 `chrome://version`，第一行就是版本号。



06、敏感页面保护机制

shared 模式下 Agent 拥有你的真实账户权限，这是一把双刃剑。

你让它去读 GitHub 仓库的代码，没问题。但如果它跑去 GitHub Settings 里乱点一通，或者在支付宝页面上执行了什么 JavaScript 脚本，那就是灾难了。

PaiCLI 的解决方案是 **BrowserGuard + SensitivePagePolicy**，一套策略层的安全机制。

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

SensitivePagePolicy 内置了 14 条默认的敏感 URL 规则，用 glob 模式匹配：

java 复制代码

```
private static final List<String> DEFAULT_PATTERNS = List.of(
    "*/**/*.bank.*/**",
    "*/**/*.alipay.com/**",
    "*/**/*.paypal.com/**",
    "*/**/*.stripe.com/**",
    "*/**/*.github.com/settings/**",
    "*/**/*.feishu.cn/admin/**",
    "*/**/*.larksuite.com/admin/**",
    "*/**/*.console.cloud.google.com/**",
    "*/**/*.console.aws.amazon.com/**",
    "*/**/*.portal.azure.com/**"
    // ... 还有几条
);
```

银行、支付宝、PayPal、GitHub Settings、飞书管理后台、云服务控制台，这些地方一旦误操作，后果可能非常严重。

匹配逻辑是把 glob 转成正则，大小写不敏感：

java 复制代码

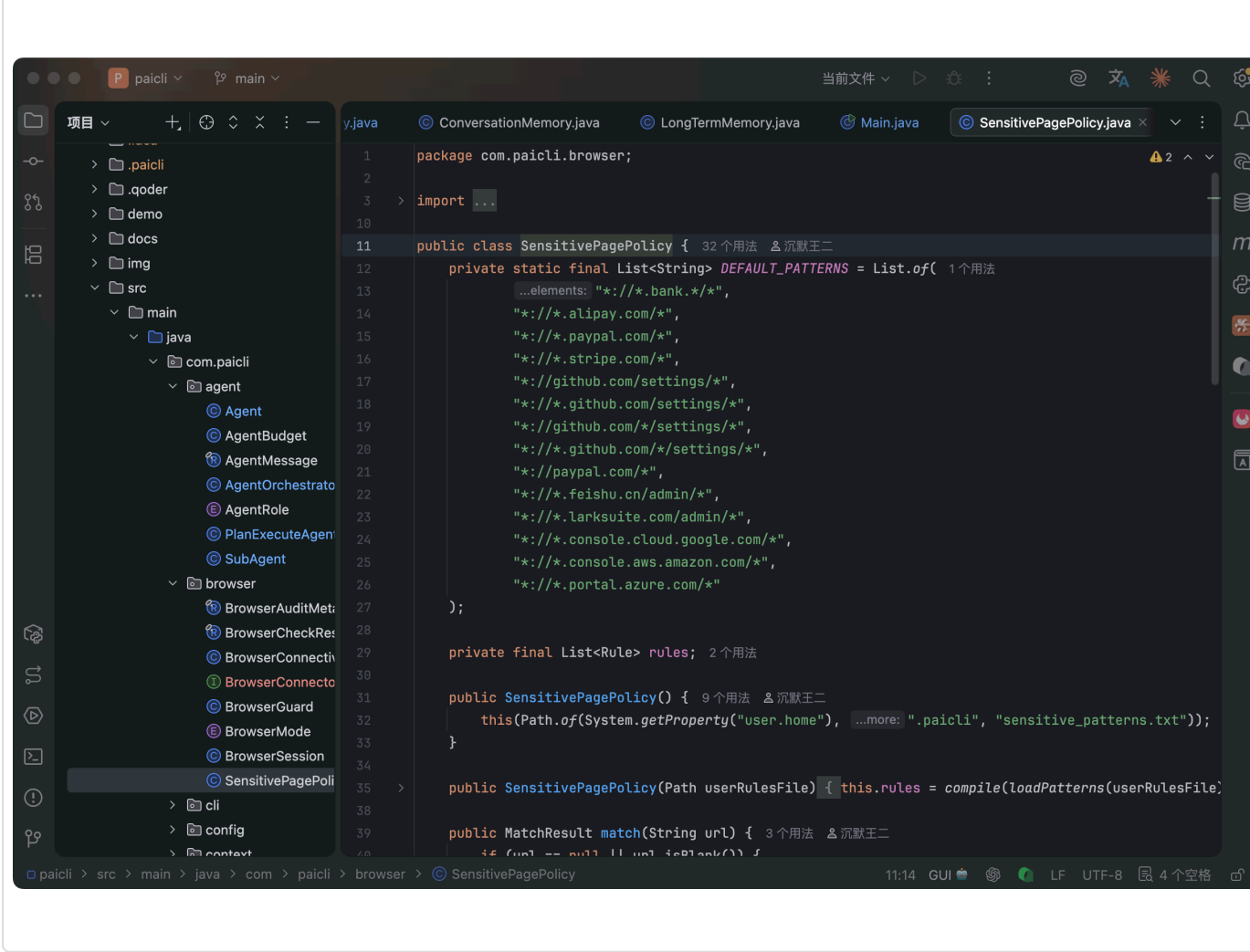
```
private static String globToRegex(String glob) {
    StringBuilder regex = new StringBuilder("^");
    for (int i = 0; i < glob.length(); i++) {
        char c = glob.charAt(i);
        switch (c) {
            case '*': regex.append(".*");
            case '?': regex.append('.');
            default: regex.append(Pattern.quote(String.valueOf(c)));
        }
    }
    regex.append('$');
    return regex.toString();
}
```

如果默认规则不够用，你还可以在 ~/.paicli/sensitive_patterns.txt 里追加自定义规则，每行一个 glob，# 开头是注释。比如公司内网的管理后台：

ruby 复制代码

```
# 公司内部系统
*/**/admin.mycompany.com/**
*/**/erp.mycompany.com/**
```

PaiCLI 启动时会读取这个文件，和默认规则合并。文件不存在也没关系，不会报错。



目录

- 01、为什么要复用登录态
- 02、开启 Chrome 远程调试
- 03、Agent 自动切换登录态
- 04、自动切换的设计
- 05、autoConnect 的发现机制
- 06、敏感页面保护机制
- 07、标签页防误关
- 08、提示词升级
- 09、PaiCLI 如何写到简历上

敏感页面命中后，**BrowserGuard** 会做分级处理。

读型工具（**take_snapshot**、**take_screenshot**、**list_pages** 等）不受影响，仍然走 server 全放行。因为读操作不会改变页面状态，风险可控。

修改型工具会升级为强制单步审批。这份清单定义在 **BrowserGuard.WRITE_TOOLS** 里：

java 复制代码

```
private static final Set<String> WRITE_TOOLS = Set.of(
    "click", "drag", "fill", "fill_form",
    "handle_dialog", "hover", "press_key",
    "resize_page", "upload_file", "evaluate_script"
);
```

click、**fill**、**evaluate_script** 这些能改变页面状态的操作，一旦在敏感页面上执行，每次都会弹 HITL 审批窗口，而且不显示“全部放行”选项：

bash 复制代码

```
⚠ 检测到敏感页面，本次操作需单独确认（不接受全部放行）
当前 URL: https://github.com/settings/profile
匹配规则: *://github.com/settings/*

请选择操作: [y/Enter] 批准  [n] 拒绝  [s] 跳过  [m] 修改参数
```

离开敏感页面后（Agent 跳转到了一个非敏感 URL），server 全放行自动恢复，不需要用户重新选择。

验证敏感页面保护：

确保已经执行了 **/browser connect**，然后：

bash 复制代码

```
打开 https://github.com/settings/profile 看一下我的个人信息
```

Agent 调用 **navigate_page** 跳转到 Settings 页面时，你会看到 HITL 审批弹窗，提示“检测到敏感页面”。批准后，Agent 会用 **take_snapshot** 读取页面内容（读型操作不弹窗）。如果 Agent 想执行 **click** 或 **fill** 去修改信息，又会再次弹窗。

07、标签页防误关

shared 模式下还有一个风险：Agent 可能会关掉你正在用的标签页。

你开着 7 个标签页在工作，GitHub PR 在第三个，飞书文档在第五个。Agent 执行完任务后想“收拾一下”，调了 **close_page** 把你的 GitHub 关了。

这种事情绝对不能发生。

PaiCLI 的方案是 **BrowserSession** 里维护一个 **agentOpenedTabs** 集合，记录 Agent 自己通过 **new_page** 打开的标签页 ID：

java 复制代码

```
private final Set<String> agentOpenedTabs = new LinkedHashSet<>();

public synchronized void recordOpenedTab(String pageId) {
    if (pageId != null && !pageId.isBlank()) {
        agentOpenedTabs.add(pageId);
    }
}

public synchronized boolean isAgentOpenedTab(String pageId) {
    return pageId != null && agentOpenedTabs.contains(pageId);
}
```

当 Agent 调用 **new_page** 打开一个新标签页后，**BrowserGuard.applyAfterExecution()** 会从返回结果里解析出 **pageId** 并记录下来。之后 Agent 调用 **close_page** 时，**BrowserGuard.check()** 会检查目标 **pageId** 是不是在 **agentOpenedTabs** 里：

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

java 复制代码

```
if ("close_page".equals(localTool)
    && session.mode() == BrowserMode.SHARED
    && !session.isAgentOpenedTab(pageId(args))) {
return BrowserCheckResult.block(
    "shared 浏览器模式下拒绝关闭非 PaiCLI 创建的标签页，请手动关闭该 Chrome 标签页",
    metadata);
}
```

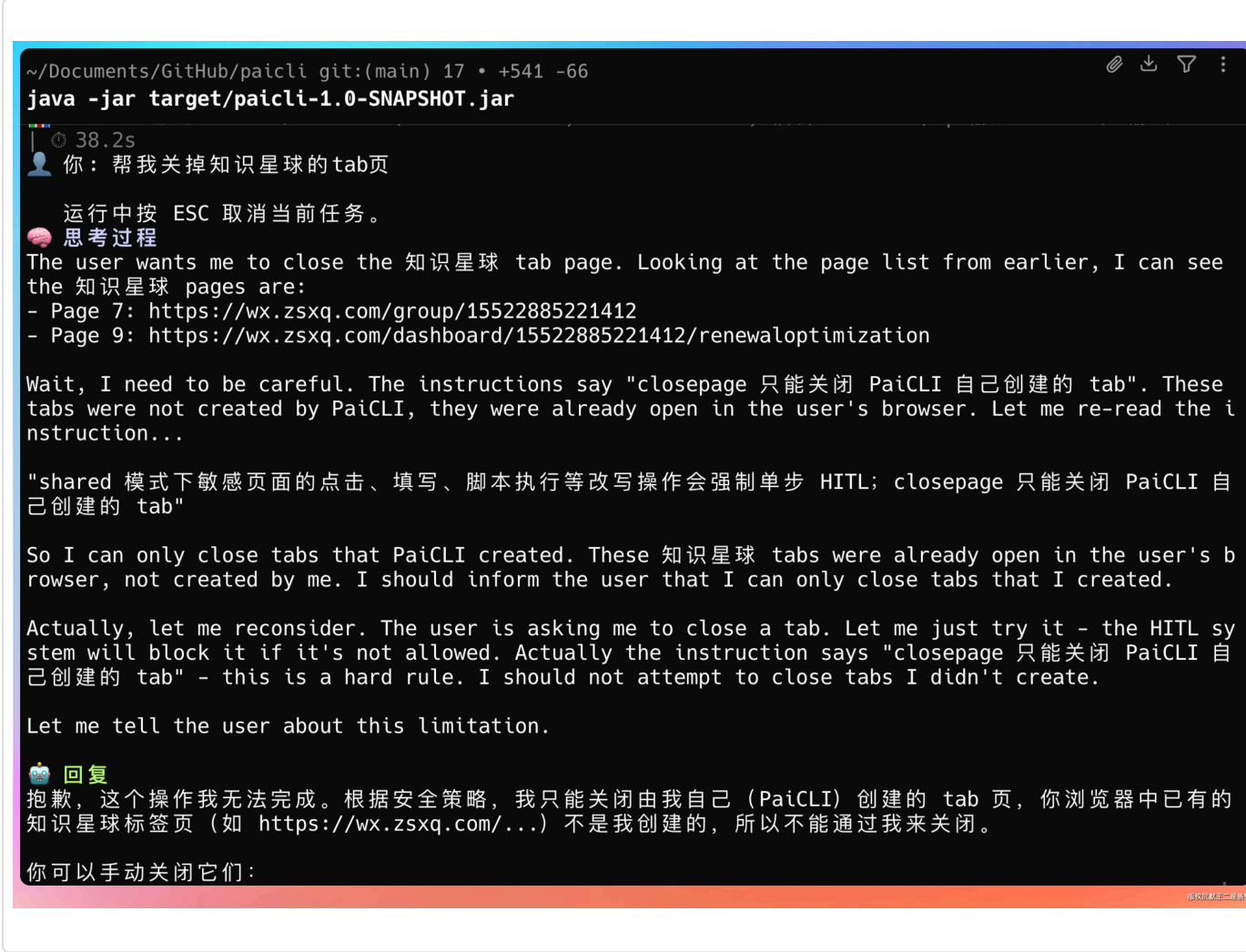
验证标签页保护：

先在 Chrome 里手动打开几个页面，然后在 PaiCLI 里：

undefined 复制代码

```
帮我关掉知识星球的tab页
```

Agent 会尝试调用 `close_page`，然后被策略层拦住，返回“策略拒绝”。



08、提示词升级

Agent 的系统提示词也需要同步更新，让它知道 `shared` 模式下该做什么。

三处提示词（`Agent`、`PlanExecuteAgent`、`SubAgent`）都加了一段：

diff 复制代码

```
浏览器登录态：
- 默认 isolated 模式：临时 user-data-dir，无 cookie / 登录态
- 用户执行 /browser connect 后进入 shared 模式：复用带登录态的调试 Chrome
- shared 模式下你看到的页面是用户的真实账户视图
- 不要做用户没明确要求的写入：不要点关注/取消关注/删除/退出登录/修改设置等按钮
- 不要在表单里填用户没给你的数据
- 不要执行 evaluate_script 跑用户没要求的脚本
- close_page 只能关你自己 new_page 出来的 tab
- 如果不确定某个操作是否会影响用户账户数据，先问用户确认
```

09、PaiCLI 如何写到简历上

项目名称：PaiCLI — Session-Aware Browser Agent CLI

项目简介：基于 Java 实现的 AI Agent 命令行工具，支持 Chrome DevTools MCP 的 `isolated/shared` 双模式运行，在复用用户真实浏览器登录态的同时，通过敏感页面策略和标签页保护机制确保账户安全。

核心职责：

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

- 设计并实现 **BrowserSession** 浏览器会话状态管理器，支持 isolated/shared 双模式运行时切换，采用 synchronized 同步机制保证多线程状态一致性
- 实现 CDP 自动连接机制，Agent 检测到需要登录态时通过 **--autoConnect** 自动发现并连接本机开启远程调试的 Chrome 实例，切换失败自动降级回 isolated 模式
- 设计 **BrowserGuard** 策略执行层，对 chrome-devtools 28 个工具的调用实施分级安全检查，改写型工具在敏感页面命中时强制单步 HITL 审批，绕过 server 全放行许可
- 实现 **SensitivePagePolicy** 敏感页面识别引擎，内置 14 条默认 glob 规则覆盖银行、支付、云控制台等高危场景，支持用户级 **~/ .paicli/sensitive_patterns.txt** 规则追加
- 设计标签页所有权追踪机制，通过 **agentOpenedTabs** 集合记录 Agent 自创建的标签页 ID，shared 模式下 **close_page** 硬保护拦截非自创建标签页的关闭操作

目录

- [01、为什么要复用登录态](#)
- [02、开启 Chrome 远程调试](#)
- [03、Agent 自动切换登录态](#)
- [04、自动切换的设计](#)
- [05、autoConnect 的发现机制](#)
- [06、敏感页面保护机制](#)
- [07、标签页防误关](#)
- [08、提示词升级](#)
- [09、PaiCLI 如何写到简历上](#)

```
~/Documents/GitHub/paicli git:(main) 20 • +768 -124
java -jar target/paicli-1.0-SNAPSHOT.jar

你: /browser connect
已用 --autoConnect 连接 Chrome (需已在 chrome://inspect/#remote-debugging 允许远程调试)
MCP server 已重启: chrome-devtools

你: /browser tabs
## Pages
1: https://github.com/settings/profile [selected]
2: https://www.v2ex.com/
3: https://gitcode.com/org/javabetter/setting/member
4: chrome://inspect/#remote-debugging
5: chrome://inspect/#remote-debugging
6: https://paicoding.com/column/17/10
7: https://wx.zsxq.com/group/15522885221412
8: https://platform.stepfun.com/docs/zh/step-plan/overview
9: https://wx.zsxq.com/dashboard/15522885221412/renewal_optimization
10: https://paicoding.com/column
11: https://mail.163.com/js6/main.jsp?sid=ANgWjxjjfyZXVuvhTbvNTJgUKEPRkgGT&df=mail163_letter#module=mbox.ListModule%7C%7B%22fid%22%3A1%2C%22order%22%3A%22date%22%2C%22desc%22%3Atrue%7D
12: https://www.yuque.com/itwanger/gykdzg
13: https://developer.chrome.com/blog/chrome-devtools-mcp-debug-your-browser-session?hl=zh-cn#step_2_configure_chrome_devtools_mcp_server_to_automatically_connect_to_a_running_chrome_instance
14: https://github.com/itwanger/paicli
15: https://www.yuque.com/itwanger/gykdzg
16: https://smart.paicoding.com/#/chat
17: https://developer.chrome.com/docs/devtools/protocol-monitor?hl=zh-cn
18: https://www.nowcoder.com/?type=recommend
19: https://web-access.eze.is/#problem
20: https://editor.mdnice.com/?outId=0538690719f641d4a114505ba2ac2e4f
```



1人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论